MANUAL PASO A PASO CON TDD

NIEVES 2048

Un juego de fusión con gráficos vistosos, construido 100% con TDD

Desarrollo Móvil Integral — Proyecto complementario de práctica TDD

🍦 11 sabores de nieve 🎨 Fichas con degradados de color 🇲🇽 Ficha final: ¡Independencia!

1. ¿Qué vamos a construir?

Nieves 2048 es un clon del famoso juego de fusión de números, pero cada valor está disfrazado de un sabor de nieve de Dolores Hidalgo: 2 es Vainilla, 4 es Fresa, 8 es Chocolate... hasta llegar a 128, que es nada menos que Mole (el sabor más icónico de la ciudad), y la ficha final, 2048, se llama '¡Independencia!' y se pinta con los colores de la bandera.

Detrás de la diversión hay un algoritmo genuinamente interesante para practicar TDD: compactar una fila, fusionar pares iguales sin fusionar tres veces seguidas, mover en las 4 direcciones reutilizando la misma lógica base, y detectar victoria o derrota. Es más algorítmico que los proyectos anteriores (Lotería, Votación, Tanda), y el resultado visual —fichas con degradados de color y animaciones de aparición— es notablemente más vistoso.

Los 11 sabores de esta versión

Valor	Sabor
2	Vainilla
4	Fresa
8	Chocolate
16	Mango
32	Limón
64	Uva
128	Mole
256	Tequila
512	Café
1024	Rompope
2048	¡Independencia! (ficha ganadora)

Reglas del juego

- Al deslizar en una dirección, todas las fichas se mueven hacia ese lado hasta topar con el borde o con otra ficha.
- Dos fichas del mismo valor que chocan se fusionan en una sola con el doble de valor (una sola vez por movimiento cada una).
- Después de cada movimiento que sí desplazó alguna ficha, aparece una nieve nueva (90% de probabilidad de Vainilla, 10% de Fresa) en una celda vacía al azar.
- Se gana al formar la ficha '¡Independencia!' (2048). Se pierde cuando la cuadrícula está llena y ya no es posible ningún movimiento ni fusión.

2. Estructura del proyecto

```
lib/
  logica/
    motor_2048.dart          (compactar, fusionar, mover una fila)
    grid.dart                (la cuadricula 4x4 completa)
    direccion.dart
    estado_juego.dart
    motor_juego.dart         (orquesta todo: puntaje, estado, fichas nuevas)
  presentation/
    juego_screen.dart        (interfaz con gradientes y animaciones)
  main.dart
test/
  motor_2048_test.dart
  grid_test.dart
  motor_juego_test.dart
```

```
flutter create nieves_2048
cd nieves_2048
```

> ✅ **BUENA PRÁCTICA —** Como en los proyectos anteriores, toda la carpeta lib/logica/ es Dart puro: ni un solo import de Flutter. Eso es lo que nos permite probar el algoritmo completo del juego en milisegundos.

RONDA 1 — Compactar una fila (quitar los huecos)

🔴 ROJO — Escribe una prueba que falle

test/motor_2048_test.dart

```
import 'package:flutter_test/flutter_test.dart';
import 'package:nieves_2048/logica/motor_2048.dart';

void main() {
  test('compactarFila quita los huecos y mantiene el orden', () {
    final resultado = Motor2048.compactarFila([2, null, 4, null]);
    expect(resultado, [2, 4, null, null]);
  });

  test('compactarFila sin huecos no cambia nada', () {
    final resultado = Motor2048.compactarFila([2, 4, 8, 16]);
    expect(resultado, [2, 4, 8, 16]);
  });
}
```

🟢 VERDE — El mínimo código para pasar

lib/logica/motor_2048.dart

```
class Motor2048 {
  static List<int?> compactarFila(List<int?> fila) {
    final valores = fila.where((v) => v != null).toList();
    while (valores.length < fila.length) {
      valores.add(null);
    }
    return valores;
  }
}
```

RONDA 2 — Fusionar pares adyacentes iguales

🔴 ROJO — Escribe una prueba que falle

test/motor_2048_test.dart (pruebas agregadas)

```
  test('fusionarFila combina un par adyacente igual', () {
    final resultado = Motor2048.fusionarFila([2, 2, 4, null]);
    expect(resultado.fila, [4, null, 4, null]);
    expect(resultado.puntosGanados, 4);
  });

  test('fusionarFila no fusiona tres iguales seguidos mas de una vez', () {
    final resultado = Motor2048.fusionarFila([2, 2, 2, null]);
    expect(resultado.fila, [4, null, 2, null]);
    expect(resultado.puntosGanados, 4);
  });
```

🟢 VERDE — El mínimo código para pasar

lib/logica/motor_2048.dart (clase y método agregados)

```
class ResultadoFusion {
  final List<int?> fila;
  final int puntosGanados;
  ResultadoFusion(this.fila, this.puntosGanados);
}

  static ResultadoFusion fusionarFila(List<int?> filaCompactada) {
    final resultado = List<int?>.from(filaCompactada);
    var puntos = 0;
    for (var i = 0; i < resultado.length - 1; i++) {
      if (resultado[i] != null && resultado[i] == resultado[i + 1]) {
        resultado[i] = resultado[i]! * 2;
        puntos += resultado[i]!;
        resultado[i + 1] = null;
      }
    }
    return ResultadoFusion(resultado, puntos);
  }
```

La segunda prueba es la que realmente importa: sin el chequeo de que resultado[i] no sea null antes de comparar, [2,2,2,null] fusionaría dos veces y produciría un resultado incorrecto (8 en vez de 4+2). Este es exactamente el tipo de caso de borde que un desarrollador olvida sin TDD.

RONDA 3 — Mover una fila completa hacia la izquierda

🔴 ROJO — Escribe una prueba que falle

test/motor_2048_test.dart (pruebas agregadas)

```
  test('mover una fila hacia la izquierda compacta y fusiona correctamente', () {
    final resultado = Motor2048.moverFilaIzquierda([null, 2, 2, 4]);
    expect(resultado.fila, [4, 4, null, null]);
    expect(resultado.puntosGanados, 4);
    expect(resultado.seMovio, true);
  });

  test('una fila ya en su posicion final reporta seMovio en falso', () {
    final resultado = Motor2048.moverFilaIzquierda([2, 4, null, null]);
    expect(resultado.seMovio, false);
  });
```

🟢 VERDE — El mínimo código para pasar

lib/logica/motor_2048.dart (clase y método agregados)

```
class ResultadoMovimiento {
  final List<int?> fila;
  final int puntosGanados;
  final bool seMovio;
  ResultadoMovimiento(this.fila, this.puntosGanados, this.seMovio);
}

  static ResultadoMovimiento moverFilaIzquierda(List<int?> filaOriginal) {
    final compactada = compactarFila(filaOriginal);
    final fusion = fusionarFila(compactada);
    final filaFinal = compactarFila(fusion.fila);

    var seMovio = false;
    for (var i = 0; i < filaOriginal.length; i++) {
      if (filaOriginal[i] != filaFinal[i]) seMovio = true;
    }
    return ResultadoMovimiento(filaFinal, fusion.puntosGanados, seMovio);
  }
```

moverFilaIzquierda no inventa lógica nueva: orquesta las dos rondas anteriores (compactar, fusionar, compactar otra vez porque fusionar deja huecos nuevos) y agrega la comparación para saber si algo cambió.

RONDA 4 — Mover la cuadrícula completa hacia la izquierda

🔴 ROJO — Escribe una prueba que falle

test/grid_test.dart

```
import 'package:flutter_test/flutter_test.dart';
import 'package:nieves_2048/logica/grid.dart';

void main() {
  test('moverIzquierda aplica el movimiento a las 4 filas', () {
    final grid = Grid([
      [null, 2, 2, null],
      [4, null, null, 4],
      [null, null, null, null],
      [2, 2, 2, 2],
    ]);

    final puntos = grid.moverIzquierda();

    expect(grid.celdas[0], [4, null, null, null]);
    expect(grid.celdas[1], [8, null, null, null]);
    expect(grid.celdas[2], [null, null, null, null]);
    expect(grid.celdas[3], [4, 4, null, null]);
    expect(puntos, 4 + 8 + 4 + 4);
  });
}
```

🟢 VERDE — El mínimo código para pasar

lib/logica/grid.dart

```
import 'motor_2048.dart';

class Grid {
  List<List<int?>> celdas;
  Grid(this.celdas);

  static Grid vacio() => Grid(List.generate(4, (_) => List.generate(4, (_) => null)));

  int moverIzquierda() {
    var puntos = 0;
    for (var f = 0; f < celdas.length; f++) {
      final resultado = Motor2048.moverFilaIzquierda(celdas[f]);
      celdas[f] = resultado.fila;
      puntos += resultado.puntosGanados;
    }
    return puntos;
  }
}
```

RONDA 5 — Mover hacia la derecha (reutilizando la lógica base)

🔴 ROJO — Escribe una prueba que falle

test/grid_test.dart (prueba agregada)

```
  test('moverDerecha empuja las fichas hacia el lado opuesto', () {
    final grid = Grid([
```

```
    [2, 2, null, null],
    [null, null, null, null],
    [null, null, null, null],
    [null, null, null, null],
  ]);

  grid.moverDerecha();

  expect(grid.celdas[0], [null, null, null, 4]);
});
```

🟢 VERDE — El mínimo código para pasar

lib/logica/grid.dart (método agregado)

```
int moverDerecha() {
  var puntos = 0;
  for (var f = 0; f < celdas.length; f++) {
    final invertida = celdas[f].reversed.toList();
    final resultado = Motor2048.moverFilaIzquierda(invertida);
    celdas[f] = resultado.fila.reversed.toList();
    puntos += resultado.puntosGanados;
  }
  return puntos;
}
```

> ✅ **BUENA PRÁCTICA —** moverDerecha no reimplementa compactar ni fusionar: invierte la fila, aplica moverFilaIzquierda, e invierte el resultado de regreso. Reutilizar en vez de duplicar es la esencia de un buen diseño.

RONDA 6 — Mover arriba y abajo (transponer la cuadrícula)

🔴 ROJO — Escribe una prueba que falle

test/grid_test.dart (pruebas agregadas)

```
  test('moverArriba empuja las fichas de cada columna hacia arriba', () {
    final grid = Grid([
      [null, null, null, null],
      [2, null, null, null],
      [null, null, null, null],
      [2, null, null, null],
    ]);

    grid.moverArriba();

    expect(grid.celdas[0][0], 4);
  });

  test('moverAbajo empuja las fichas de cada columna hacia abajo', () {
    final grid = Grid([
      [2, null, null, null],
      [null, null, null, null],
      [null, null, null, null],
      [null, null, null, null],
    ]);

    grid.moverAbajo();

    expect(grid.celdas[3][0], 2);
  });
```

🟢 VERDE — El mínimo código para pasar

lib/logica/grid.dart (métodos agregados)

```
  List<List<int?>> _transponer(List<List<int?>> g) {
    final n = g.length;
    return List.generate(n, (i) => List.generate(n, (j) => g[j][i]));
  }

  int moverArriba() {
    celdas = _transponer(celdas);
    final puntos = moverIzquierda();
    celdas = _transponer(celdas);
    return puntos;
  }

  int moverAbajo() {
    celdas = _transponer(celdas);
    final puntos = moverDerecha();
    celdas = _transponer(celdas);
    return puntos;
  }
```

Este es el momento más elegante del proyecto: en vez de escribir un cuarto algoritmo de movimiento vertical desde cero, transponemos la cuadrícula (convertimos filas en columnas), reutilizamos moverIzquierda/moverDerecha —que ya mueven filas horizontalmente— y transponemos de regreso. Cuatro direcciones, un solo algoritmo real.

RONDA 7 — Agregar una ficha nueva al azar

🔴 ROJO — Escribe una prueba que falle

test/grid_test.dart (pruebas agregadas)

```
import 'dart:math';

  test('agregarFichaAleatoria coloca exactamente una ficha nueva', () {
    final grid = Grid.vacio();

    grid.agregarFichaAleatoria(aleatorio: Random(1));

    final noVacias = grid.celdas.expand((f) => f).where((v) => v != null).length;
    expect(noVacias, 1);
  });

  test('agregarFichaAleatoria no hace nada si la cuadricula esta llena', () {
    final grid = Grid(List.generate(4, (_) => List.generate(4, (_) => 2)));

    grid.agregarFichaAleatoria(aleatorio: Random(1));

    final valores = grid.celdas.expand((f) => f).toList();
    expect(valores.every((v) => v == 2), true);
  });
```

🟢 VERDE — El mínimo código para pasar

lib/logica/grid.dart (método agregado)

```
  void agregarFichaAleatoria({Random? aleatorio}) {
    final rnd = aleatorio ?? Random();
    final vacias = <List<int>>[];
    for (var f = 0; f < celdas.length; f++) {
      for (var c = 0; c < celdas[f].length; c++) {
        if (celdas[f][c] == null) vacias.add([f, c]);
      }
    }
    if (vacias.isEmpty) return;

    final elegida = vacias[rnd.nextInt(vacias.length)];
    final valor = rnd.nextDouble() < 0.9 ? 2 : 4;
    celdas[elegida[0]][elegida[1]] = valor;
  }
```

> ⚠ **EVITA ESTO —** El parámetro aleatorio es otra 'costura' de diseño, igual que los constructores desdeCartas y desdeFilas del proyecto de la Lotería: sin él, esta función sería imposible de probar de forma determinista, porque cada corrida elegiría una celda distinta.

RONDA 8 — Detectar si ya no hay movimientos posibles

🔴 ROJO — Escribe una prueba que falle

test/grid_test.dart (pruebas agregadas)

```
  test('hayMovimientosPosibles es verdadero si hay una celda vacia', () {
    final grid = Grid.vacio();
    expect(grid.hayMovimientosPosibles(), true);
  });

  test('hayMovimientosPosibles es verdadero si hay una fusion posible', () {
    final grid = Grid([
      [2, 2, 4, 8],
      [4, 8, 16, 32],
      [2, 4, 8, 16],
      [4, 2, 4, 2],
    ]);
    expect(grid.hayMovimientosPosibles(), true); // 2,2 en la esquina superior
  });

  test('hayMovimientosPosibles es falso si la cuadricula esta llena y sin fusiones', () {
    final grid = Grid([
      [2, 4, 2, 4],
      [4, 2, 4, 2],
      [2, 4, 2, 4],
      [4, 2, 4, 2],
    ]);
    expect(grid.hayMovimientosPosibles(), false);
  });
```

🟢 VERDE — El mínimo código para pasar

lib/logica/grid.dart (métodos agregados)

```
  bool hayCeldasVacias() => celdas.expand((f) => f).any((v) => v == null);

  bool hayMovimientosPosibles() {
    if (hayCeldasVacias()) return true;

    for (var f = 0; f < celdas.length; f++) {
      for (var c = 0; c < celdas[f].length - 1; c++) {
        if (celdas[f][c] == celdas[f][c + 1]) return true;
      }
    }
    for (var c = 0; c < celdas.length; c++) {
      for (var f = 0; f < celdas.length - 1; f++) {
        if (celdas[f][c] == celdas[f + 1][c]) return true;
      }
    }
    return false;
  }
```

Revisamos tres cosas en orden: ¿hay una celda vacía? ¿hay dos vecinos horizontales iguales? ¿hay dos vecinos verticales iguales? Si las tres respuestas son 'no', el juego realmente terminó.

RONDA 9 — Detectar la victoria

🔴 ROJO — Escribe una prueba que falle

test/grid_test.dart (prueba agregada)

```
  test('haGanado es verdadero cuando una celda alcanza el objetivo', () {
    final grid = Grid.vacio();
    grid.celdas[0][0] = 2048;

    expect(grid.haGanado(2048), true);
  });

  test('haGanado es falso si ninguna celda alcanza el objetivo', () {
    final grid = Grid.vacio();
    grid.celdas[0][0] = 1024;

    expect(grid.haGanado(2048), false);
  });
```

🟢 VERDE — El mínimo código para pasar

lib/logica/grid.dart (método agregado)

```
  bool haGanado(int objetivo) =>
      celdas.expand((f) => f).any((v) => v != null && v >= objetivo);
```

RONDA 10 — MotorJuego: integrar todo (puntaje, turnos, victoria/derrota)

🔴 ROJO — Escribe una prueba que falle

test/motor_juego_test.dart

```
import 'package:flutter_test/flutter_test.dart';
import 'package:nieves_2048/logica/grid.dart';
import 'package:nieves_2048/logica/direccion.dart';
import 'package:nieves_2048/logica/estado_juego.dart';
import 'package:nieves_2048/logica/motor_juego.dart';

void main() {
  test('mover sin ningun desplazamiento no agrega una ficha nueva', () {
    final motor = MotorJuego();
    motor.grid = Grid([
      [2, 4, 8, 16],
      [null, null, null, null],
      [null, null, null, null],
      [null, null, null, null],
    ]);
    final vaciasAntes = motor.grid.celdas.expand((f) => f).where((v) => v == null).length;

    motor.mover(Direccion.izquierda); // la fila ya esta compactada, sin fusiones posibles

    final vaciasDespues = motor.grid.celdas.expand((f) => f).where((v) => v == null).length;
    expect(vaciasDespues, vaciasAntes);
  });

  test('alcanzar el valor objetivo cambia el estado a ganado', () {
    final motor = MotorJuego();
    motor.grid = Grid([
      [1024, 1024, null, null],
      [null, null, null, null],
      [null, null, null, null],
      [null, null, null, null],
    ]);

    motor.mover(Direccion.izquierda);

    expect(motor.estado, EstadoJuego.ganado);
    expect(motor.puntaje, 2048);
  });
}
```

🟢 VERDE — El mínimo código para pasar

lib/logica/direccion.dart

```
enum Direccion { izquierda, derecha, arriba, abajo }
```

lib/logica/estado_juego.dart

```
enum EstadoJuego { jugando, ganado, perdido }
```

lib/logica/motor_juego.dart

```dart
import 'grid.dart';
import 'direccion.dart';
import 'estado_juego.dart';

class MotorJuego {
  Grid grid = Grid.vacio();
  int puntaje = 0;
  EstadoJuego estado = EstadoJuego.jugando;
  static const int objetivoVictoria = 2048;

  MotorJuego() {
    grid.agregarFichaAleatoria();
    grid.agregarFichaAleatoria();
  }

  void mover(Direccion direccion) {
    if (estado != EstadoJuego.jugando) return;

    final antes = grid.celdas.map((f) => List<int?>.from(f)).toList();

    int puntosGanados;
    switch (direccion) {
      case Direccion.izquierda:
        puntosGanados = grid.moverIzquierda();
        break;
      case Direccion.derecha:
        puntosGanados = grid.moverDerecha();
        break;
      case Direccion.arriba:
        puntosGanados = grid.moverArriba();
        break;
      case Direccion.abajo:
        puntosGanados = grid.moverAbajo();
        break;
    }
    puntaje += puntosGanados;

    if (grid.haGanado(objetivoVictoria)) {
      estado = EstadoJuego.ganado;
      return;
    }

    final huboMovimiento = !_gridsIguales(antes, grid.celdas);
    if (huboMovimiento) {
      grid.agregarFichaAleatoria();
    }

    if (!grid.hayMovimientosPosibles()) {
      estado = EstadoJuego.perdido;
    }
  }

  bool _gridsIguales(List<List<int?>> a, List<List<int?>> b) {
    for (var f = 0; f < a.length; f++) {
      for (var c = 0; c < a[f].length; c++) {
        if (a[f][c] != b[f][c]) return false;
      }
    }
    return true;
  }
}
```

> ✅ **BUENA PRÁCTICA —** MotorJuego compara el estado 'antes' y 'después' del movimiento para decidir si agregar una ficha nueva — así replicamos la regla real de 2048: si el movimiento no desplazó nada, tampoco debe aparecer una ficha nueva de la nada.

4. Prueba de integración: una partida completa

test/motor_juego_test.dart (prueba agregada)

```
  test('una serie de movimientos combina fichas y acumula puntaje', () {
    final motor = MotorJuego();
    motor.grid = Grid([
      [2, 2, null, null],
      [null, null, null, null],
      [null, null, null, null],
      [null, null, null, null],
    ]);

    motor.mover(Direccion.izquierda); // fusiona en 4, aparece una ficha nueva

    expect(motor.grid.celdas[0][0], 4);
    expect(motor.puntaje, 4);
    expect(motor.estado, EstadoJuego.jugando);
  });
```

Con las 10 rondas anteriores ya probadas, esta prueba de integración confirma que el ciclo completo —mover, fusionar, sumar puntos, agregar ficha nueva, verificar estado— funciona correctamente en conjunto.

5. La interfaz gráfica: degradados de color y animaciones

Cada sabor de nieve tiene su propio degradado de color, y cada ficha usa AnimatedSwitcher para 'aparecer' con una animación de escala cada vez que su valor cambia (ya sea porque nació o porque se fusionó). El juego se controla deslizando el dedo en cualquier dirección.

lib/presentation/juego_screen.dart (ficha visual)

```
import 'package:flutter/material.dart';

class Ficha extends StatelessWidget {
  final int? valor;
  const Ficha({super.key, required this.valor});

  static const Map<int, List<Color>> _gradientes = {
    2: [Color(0xFFFFFDE7), Color(0xFFFFF9C4)],
    4: [Color(0xFFF8BBD0), Color(0xFFF48FB1)],
    8: [Color(0xFFD7CCC8), Color(0xFFA1887F)],
    16: [Color(0xFFFFE0B2), Color(0xFFFFB74D)],
    32: [Color(0xFFC8E6C9), Color(0xFF81C784)],
    64: [Color(0xFFE1BEE7), Color(0xFFBA68C8)],
    128: [Color(0xFF8D6E63), Color(0xFF4E342E)],
    256: [Color(0xFFFFE082), Color(0xFFFFC107)],
    512: [Color(0xFF6D4C41), Color(0xFF3E2723)],
    1024: [Color(0xFFFFF3E0), Color(0xFFFFCC80)],
    2048: [Color(0xFF006341), Color(0xFFFFFFFF)],
  };

  static const Map<int, String> _nombres = {
    2: 'Vainilla', 4: 'Fresa', 8: 'Chocolate', 16: 'Mango',
    32: 'Limon', 64: 'Uva', 128: 'Mole', 256: 'Tequila',
    512: 'Cafe', 1024: 'Rompope', 2048: 'Independencia',
  };

  @override
  Widget build(BuildContext context) {
    final colores = _gradientes[valor] ?? [Colors.grey.shade200, Colors.grey.shade300];
```

```
    return AnimatedSwitcher(
      duration: const Duration(milliseconds: 250),
      transitionBuilder: (child, animacion) => ScaleTransition(scale: animacion, child: child),
      child: Container(
        key: ValueKey(valor),
        decoration: BoxDecoration(
          gradient: LinearGradient(colors: colores, begin: Alignment.topLeft, end:
Alignment.bottomRight),
          borderRadius: BorderRadius.circular(12),
          boxShadow: valor == null
              ? []
              : [BoxShadow(color: Colors.black.withOpacity(0.15), blurRadius: 4, offset: const
Offset(0, 2))],
        ),
        alignment: Alignment.center,
        child: valor == null
            ? null
            : Column(
                mainAxisSize: MainAxisSize.min,
                children: [
                  Text(
                    valor.toString(),
                    style: TextStyle(
                      fontSize: valor >= 128 ? 18 : 22,
                      fontWeight: FontWeight.bold,
                      color: valor >= 128 ? Colors.white : Colors.black87,
                    ),
                  ),
                  Text(
                    _nombres[valor] ?? '',
                    style: TextStyle(fontSize: 10, color: valor >= 128 ? Colors.white70 :
Colors.black54),
                  ),
                ],
              ),
      ),
    );
  }
}
```

lib/presentation/juego_screen.dart (pantalla principal)

```
class JuegoScreen extends StatefulWidget {
  const JuegoScreen({super.key});
  @override
  State<JuegoScreen> createState() => _JuegoScreenState();
}

class _JuegoScreenState extends State<JuegoScreen> {
  late MotorJuego _motor;

  @override
  void initState() {
    super.initState();
    _motor = MotorJuego();
  }

  void _mover(Direccion direccion) {
    setState(() => _motor.mover(direccion));
    if (_motor.estado != EstadoJuego.jugando) _mostrarResultado();
  }
```

```
  void _mostrarResultado() {
    final gano = _motor.estado == EstadoJuego.ganado;
    showGeneralDialog(
      context: context,
      barrierDismissible: true,
      barrierLabel: 'resultado',
      transitionDuration: const Duration(milliseconds: 450),
      pageBuilder: (context, anim1, anim2) => const SizedBox.shrink(),
      transitionBuilder: (context, anim1, anim2, child) {
        return ScaleTransition(
          scale: CurvedAnimation(parent: anim1, curve: Curves.elasticOut),
          child: FadeTransition(
            opacity: anim1,
            child: AlertDialog(
              title: Text(gano ? 'Independencia lograda!' : 'Se acabaron los movimientos'),
              content: Text('Puntaje final: ' + _motor.puntaje.toString()),
              actions: [
                TextButton(
                  onPressed: () {
                    Navigator.pop(context);
                    setState(() => _motor = MotorJuego());
                  },
                  child: const Text('Jugar de nuevo'),
                ),
              ],
            ),
          ),
        );
      },
    );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Nieves 2048 - Dolores Hidalgo')),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(16),
            child: TweenAnimationBuilder<int>(
              tween: IntTween(begin: 0, end: _motor.puntaje),
              duration: const Duration(milliseconds: 400),
              builder: (context, valor, _) => Text(
                'Puntaje: ' + valor.toString(),
                style: const TextStyle(fontSize: 22, fontWeight: FontWeight.bold),
              ),
            ),
          ),
          Expanded(
            child: GestureDetector(
              onHorizontalDragEnd: (detalle) {
                final v = detalle.primaryVelocity ?? 0;
                if (v > 0) _mover(Direccion.derecha);
                if (v < 0) _mover(Direccion.izquierda);
              },
              onVerticalDragEnd: (detalle) {
                final v = detalle.primaryVelocity ?? 0;
                if (v > 0) _mover(Direccion.abajo);
                if (v < 0) _mover(Direccion.arriba);
              },
              child: Padding(
                padding: const EdgeInsets.all(12),
                child: GridView.builder(
```

```
                physics: const NeverScrollableScrollPhysics(),
                gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
                  crossAxisCount: 4,
                  crossAxisSpacing: 8,
                  mainAxisSpacing: 8,
                ),
                itemCount: 16,
                itemBuilder: (context, i) {
                  final valor = _motor.grid.celdas[i ~/ 4][i % 4];
                  return Ficha(valor: valor);
                },
              ),
            ),
          ),
        ),
        const Padding(
          padding: EdgeInsets.all(12),
          child: Text('Desliza en cualquier direccion para mover las nieves'),
        ),
      ],
    ),
  );
 }
}
```

✅ **BUENA PRÁCTICA —** Fíjate que JuegoScreen nunca decide qué pasa cuando dos fichas chocan, ni cuándo se gana o se pierde — todo eso ya vive, probado, en MotorJuego. La pantalla solo traduce gestos de deslizamiento en llamadas a mover() y dibuja lo que MotorJuego reporta.

lib/main.dart

```
import 'package:flutter/material.dart';
import 'presentation/juego_screen.dart';

void main() => runApp(const Nieves2048App());

class Nieves2048App extends StatelessWidget {
  const Nieves2048App({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Nieves 2048',
      theme: ThemeData(primarySwatch: Colors.pink, useMaterial3: true),
      home: const JuegoScreen(),
    );
  }
}
```

6. Cómo ejecutar todo

1. Ejecuta flutter test — deberían pasar 15 pruebas en verde, repartidas en los 3 archivos de prueba.
2. Ejecuta flutter run.
3. Desliza el dedo sobre la cuadrícula en cualquier dirección para mover las nieves.
4. Intenta llegar hasta la ficha '¡Independencia!' (2048) y observa la animación final.

7. Checklist de este proyecto TDD

✓	Verificación
☐	compactarFila y fusionarFila se probaron por separado antes de combinarse en moverFilaIzquierda
☐	Existe una prueba específica para el caso de tres fichas iguales seguidas (no se fusionan dos veces)
☐	moverDerecha, moverArriba y moverAbajo reutilizan moverIzquierda en vez de duplicar el algoritmo
☐	agregarFichaAleatoria acepta un Random inyectado para pruebas deterministas
☐	MotorJuego decide agregar una ficha nueva solo si el movimiento realmente desplazó algo
☐	La interfaz no reimplementa ninguna regla del juego — solo traduce gestos y dibuja el estado

8. Retos de extensión

> 🎯 **RETO —** Agrega una animación de 'temblor' (shake) cuando el jugador intenta deslizar en una dirección donde ningún movimiento es posible, para dar retroalimentación visual inmediata.

- Guarda el puntaje más alto (record) de forma persistente con shared_preferences.
- Agrega un modo de dificultad donde la cuadrícula sea de 5x5 en lugar de 4x4.
- Cambia la probabilidad de aparición de fichas (por ejemplo, 80/20 en vez de 90/10) y ajusta las pruebas correspondientes.
- Agrega un botón de 'deshacer' el último movimiento, guardando el estado anterior del grid.

🏆 **INSIGNIA DESBLOQUEADA: Cinturón Negro de Proyecto — construiste el algoritmo completo de un juego de fusión, con gráficos vistosos, 100% guiado por pruebas**